S.I.G.S.M.

Sistema Integral de Gestión de Servicios Médicos

MANUAL DE INSTALACIÓN Y MANTENIMIENTO

1. Introducción y alcance..... 4

2. Descripción general y arquitectura..... 4

2.1 Componentes del sistema.....4

2.2 Persistencia de información..... 5

2.3 Organización del despliegue.....5

3. Requisitos previos..... 5

3.1 Software y recursos.....5

3.2 Comprobación de herramientas.....6

4. Procedimiento de instalación..... 6

4.1 Obtener el código fuente.....6

4.2 Crear el archivo de configuración.....7

4.3 Validar la configuración..... 8

4.4 Instalar la aplicación.....8

4.5 Inicialización de base de datos y administrador.....9

5. Verificación y puesta en funcionamiento..... 9

5.1 Estado de servicios..... 9

5.2 Consultar registros..... 10

5.3 Prueba funcional..... 10

5.4 Identificar la versión instalada..... 10

6. Operación y administración de servicios..... 11

6.1 Comandos habituales..... 11

6.2 Acceder a los contenedores..... 11

7. Actualización y mantenimiento preventivo..... 12

7.1 Actualización controlada..... 12

7.2 Mantenimiento preventivo..... 13

8. Política de respaldos y recuperación.....13

8.1 Información que debe protegerse..... 13

8.2 Respaldo manual de MariaDB..... 14
docker compose exec db sh -c 'MYSQL_PWD="$MARIADB_ROOT_PASSWORD" mariadb-dump -uroot --single-transaction --quick --routines --triggers sigsm > /tmp/sigsm_backup.sql'..... 14
docker compose cp db:/tmp/sigsm_backup.sql ./sigsm_backup.sql..... 14
docker compose exec db rm /tmp/sigsm_backup.sql..... 14
8.3 Respaldo de documentos..... 14
8.4 Automatización existente en el proyecto..... 15
8.5 Restauración..... 15
9. Seguridad y acceso a la aplicación..... 16
9.1 Acceso local y por red..... 16
9.2 Limitaciones del despliegue..... 16
9.3 Protección de secretos y permisos..... 16
10. Resolución de problemas frecuentes..... 17
10.1 Docker no responde o la aplicación no abre..... 17
10.2 Error 502 en la API..... 17
10.3 Fallos de autenticación o admin-init..... 17
10.4 MariaDB no saludable o esquema sin actualizar..... 18
10.5 Seed de demostración o PDF faltantes..... 18
10.6 Los cambios publicados no aparecen..... 18
11. Lista de comprobación final..... 19
12. Referencias..... 19

1. Introducción y alcance

El presente documento establece el procedimiento técnico para instalar, configurar, poner en funcionamiento y mantener S.I.G.S.M., una aplicación web modular desarrollada por Empre S.A.S. Está dirigido a docentes evaluadores, integrantes del equipo y responsables técnicos que necesiten reproducir el entorno y administrar la solución.

S.I.G.S.M. centraliza tareas de gestión documental, encuestas, autenticación de funcionarios y administración y seguimiento de traslados. Su caso de estudio es el Hospital de Clínicas de Montevideo. La aplicación publicada en el repositorio es un proyecto académico: este manual no acredita un despliegue oficial ni autoriza su utilización con datos reales de pacientes.

Se adopta Docker Compose como método recomendado de instalación. Esta modalidad evita configurar manualmente las dependencias del frontend, backend y base de datos en el equipo anfitrión, facilita la reproducibilidad del entorno y separa la aplicación de su información persistente.

2. Descripción general y arquitectura

2.1 Componentes del sistema

El frontend está desarrollado en React y Vite, utiliza Material UI y es servido por Nginx. El backend es una API REST en PHP 8.2 ejecutada mediante Apache. La base de datos emplea MariaDB 11.4. Docker Compose administra estos tres servicios principales y dos tareas auxiliares: admin-init, responsable de generar la cuenta administradora inicial, y seed-demo, destinado exclusivamente a introducir datos ficticios cuando se activa el perfil demo.

Flujo de comunicación: navegador → frontend/Nginx → solicitudes /api hacia backend/Apache-PHP → MariaDB. El servicio frontend es el único que publica un puerto del anfitrión; el backend y MariaDB permanecen accesibles solamente en la red interna de Docker.

2.2 Persistencia de información

La configuración declara dos volúmenes: db_data, montado en /var/lib/mysql para la base de datos, y docs_data, montado en /var/www/api/storage para los archivos subidos. Docker suele denominarlos sigsm_db_data y sigsm_docs_data. Separar los volúmenes de los contenedores permite detener, eliminar o reconstruir servicios sin perder los datos, siempre que no se eliminen dichos volúmenes.

2.3 Organización del despliegue

La carpeta de referencia es prototipos/docker. Contiene compose.yaml, .env.example, README.md y los subdirectorios backend, database y frontend con sus Dockerfiles y configuraciones auxiliares. Los Dockerfiles obtienen el código de la rama main del repositorio durante la construcción. Por ello, las modificaciones locales de los proyectos no se incorporan automáticamente a las imágenes.

3. Requisitos previos

3.1 Software y recursos

Se requiere Docker Engine o Docker Desktop con Docker Compose v2, Git, una terminal con acceso a los comandos, conexión a Internet para descargar imágenes y consultar GitHub durante la construcción, un puerto libre para el frontend y capacidad de disco suficiente para imágenes, base de datos y documentos. En Windows debe estar iniciado Docker Desktop; en Linux, el servicio Docker debe encontrarse activo y el usuario debe disponer de permisos.

No es necesario instalar PHP, Apache, Node.js, Nginx, MariaDB ni XAMPP de manera independiente en la computadora utilizada para el despliegue con Docker.

3.2 Comprobación de herramientas

```shell
docker --version
docker compose version
docker info
git --version
```

Los comandos deben mostrar las versiones instaladas y la información del motor Docker. Si docker info no responde, primero se debe solucionar el problema de arranque o permisos del motor.

4. Procedimiento de instalación

4.1 Obtener el código fuente

Abrir una terminal y ejecutar los siguientes comandos. El repositorio oficial utilizado por este manual es https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU.

```shell
git clone https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU.git
cd S.I.G.S.M---Proyecto-UTU/prototipos/docker
```

Como alternativa para un equipo que únicamente ejecutará la aplicación, se puede realizar una descarga parcial mediante Git sparse-checkout:

```shell
git clone --depth 1 --filter=blob:none --sparse --branch main https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU.git
cd S.I.G.S.M---Proyecto-UTU
git sparse-checkout set prototipos/docker
cd prototipos/docker
```

Todos los comandos de Docker Compose indicados en adelante se ejecutan desde prototipos/docker. El código de la API, frontend y SQL se obtiene automáticamente durante la construcción de las imágenes.

4.2 Crear el archivo de configuración

Copiar .env.example con el nombre .env. En Linux/macOS ejecutar cp .env.example .env; en Windows PowerShell ejecutar Copy-Item .env.example .env. A continuación, editar el archivo para definir la configuración del despliegue.

```none
WEB_PORT=8080
DB_USER=sigsm_app
DB_PASSWORD=REEMPLAZAR_POR_CLAVE_SEGURA
MARIADB_ROOT_PASSWORD=REEMPLAZAR_POR_OTRA_CLAVE_SEGURA
ADMIN_NAME="Administrador Inicial"
ADMIN_EMAIL=administrador@ejemplo.com
ADMIN_PASSWORD=REEMPLAZAR_POR_CLAVE_DE_12_CARACTERES_O_MAS
```

WEB_PORT publica la interfaz; DB_USER y DB_PASSWORD corresponden a la cuenta interna de la base de datos; MARIADB_ROOT_PASSWORD es la clave de administración de MariaDB; ADMIN_NAME, ADMIN_EMAIL y ADMIN_PASSWORD permiten generar la primera cuenta de la aplicación. La contraseña de dicha cuenta

debe tener al menos 12 caracteres. Los valores mostrados son ejemplos, no credenciales utilizables.

El archivo .env contiene secretos: debe mantenerse fuera del repositorio, no compartirse mediante capturas y almacenarse con permisos adecuados. Cambiar las contraseñas en .env después de inicializar el volumen no actualiza automáticamente los usuarios ya existentes en MariaDB o en la aplicación.

4.3 Validar la configuración

```shell
docker compose config --quiet
```

Si el comando termina sin errores, Compose reconoce la configuración y las variables requeridas. La opción --quiet evita mostrar el contenido completo de la configuración, que podría incluir información sensible.

4.4 Instalar la aplicación

Para una instalación habitual, sin datos ficticios adicionales, ejecutar:

```shell
docker compose up -d --build
```

Compose construye las imágenes, crea la red interna y los volúmenes, inicializa MariaDB, ejecuta los esquemas SQL de la base cuando el volumen está vacío, crea el administrador inicial y finalmente inicia backend y frontend. La opción -d inicia los servicios en segundo plano.

En una instalación nueva de prueba, si se necesitan registros ficticios para presentar y comprobar la aplicación, se puede activar el perfil demo:

```shell
docker compose --profile demo up -d --build
```

El servicio seed-demo importa prototipos/backend/database/data/seedDemo.sql después de la creación del administrador. Está preparado para una instalación limpia y no debe ejecutarse repetidamente sobre una base que ya contenga información. Los PDF no se generan con el seed, puesto que corresponden a archivos físicos que deben cargarse por separado.

4.5 Inicialización de base de datos y administrador

Durante el primer arranque, la imagen de MariaDB ejecuta en orden dbSchema_01.sql, dbSchema_02.sql, dbSchema_03.sql y dbSchema_04.sql. Estos scripts solamente se ejecutan automáticamente cuando el directorio de datos está vacío. Si ya existe el volumen db_data, reconstruir una imagen no equivale a migrar el esquema de una base existente.

El servicio admin-init comprueba el rol Administrador, crea la cuenta asociada a ADMIN_EMAIL si no existe, almacena la contraseña mediante hash y asigna el rol correspondiente. Si el administrador ya existe, no sustituye automáticamente su contraseña.

5. Verificación y puesta en funcionamiento

5.1 Estado de servicios

```shell
docker compose ps
```

```
docker compose ps -a
```

Los servicios frontend, backend y db deben permanecer activos. MariaDB debe alcanzar el estado healthy. admin-init y seed-demo son tareas de ejecución puntual y pueden figurar como finalizadas correctamente.

5.2 Consultar registros

```Shell
docker compose logs --tail=100 frontend
docker compose logs --tail=100 backend
docker compose logs --tail=100 db
docker compose logs admin-init
docker compose --profile demo logs seed-demo
```

Los registros permiten identificar errores de inicialización, problemas de conexión o fallas en la creación del administrador. Para seguimiento continuo, agregar -f a docker compose logs y salir con Ctrl+C; esta acción no detiene los contenedores.

5.3 Prueba funcional

Abrir http://localhost:8080 o el puerto definido en WEB_PORT. Confirmar que se muestre la pantalla de inicio de sesión y acceder con ADMIN_EMAIL y ADMIN_PASSWORD configurados para la instalación inicial. Verificar la navegación de los módulos habilitados, la lectura de categorías y documentos, y la consulta de traslados. Si se utilizó el perfil demo, comprobar que se visualicen los registros ficticios esperados.

5.4 Identificar la versión instalada

```shell
docker compose exec frontend cat /etc/sigsm-commit
docker compose exec backend cat /etc/sigsm-commit
docker compose exec db cat /etc/sigsm-commit
```

Estos comandos muestran el identificador del commit incluido en cada imagen. Conviene registrarlo en el historial de mantenimiento para saber qué versión se ejecuta y facilitar la recuperación ante incidentes.

6. Operación y administración de servicios

6.1 Comandos habituales

```shell
docker compose up -d          # iniciar
docker compose stop           # detener sin borrar
docker compose start          # volver a iniciar
docker compose restart        # reiniciar
docker compose ps             # estado
docker stats                  # consumo de recursos
docker compose down                 # quitar contenedores y red, conservando volúmenes
```

Para reiniciar solo un componente puede utilizarse docker compose restart frontend, backend o db. Reiniciar un contenedor no incorpora código nuevo desde GitHub. El comando docker compose down no debe confundirse con docker compose down -v: este último elimina también los volúmenes persistentes y destruye la base de datos y los documentos del proyecto.

6.2 Acceder a los contenedores

```shell
docker compose exec backend bash
docker compose exec frontend sh
docker compose exec db sh
```

Estas terminales se utilizan exclusivamente para diagnóstico o mantenimiento autorizado.

7. Actualización y mantenimiento preventivo

7.1 Actualización controlada

Antes de actualizar, registrar la versión actual, revisar los cambios publicados en main y realizar un respaldo verificable. Para cambios del frontend, reconstruir solo ese servicio; para cambios PHP, reconstruir backend. Como los Dockerfiles descargan código de GitHub durante el build, una simple operación de restart no cambia la versión.

```shell
docker compose build --no-cache frontend
docker compose up -d --no-deps frontend
```

Para actualizar exclusivamente el backend:

```shell
docker compose build --no-cache backend
docker compose up -d --no-deps backend
```

Para cambios en los archivos de Docker Compose, ejecutar primero git pull origin main desde el directorio del repositorio y después reconstruir y levantar los contenedores necesarios:

```shell
git pull origin main
docker compose build --no-cache
docker compose up -d
```

En despliegues con cambios en base de datos se requiere una migración SQL diseñada y probada. No se deben volver a ejecutar los scripts de creación inicial sobre una instalación poblada ni eliminar volúmenes para forzar la actualización.

7.2 Mantenimiento preventivo

El mantenimiento comprende revisar periódicamente el estado de los contenedores, los mensajes de error, el espacio disponible, el uso de memoria y CPU, la vigencia de imágenes y dependencias, la ejecución de respaldos y la accesibilidad de las funciones esenciales. Conviene registrar fecha, responsable, versión, incidencia y acción realizada en una bitácora de mantenimiento.

Como criterio operativo propuesto, realizar una revisión semanal del estado y los registros, una comprobación semanal del resultado de respaldos y una prueba de restauración periódica en un entorno aislado. Estas frecuencias son recomendaciones de administración, no tareas programadas automáticamente por Compose.

8. Política de respaldos y recuperación

8.1 Información que debe protegerse

Es imprescindible respaldar conjuntamente la base de datos de MariaDB y los archivos del volumen docs_data. La primera almacena los registros y metadatos; el segundo conserva el contenido físico de documentos. Respaldar únicamente uno de

estos componentes puede producir referencias a archivos inexistentes o archivos sin registro asociado.

8.2 Respaldo manual de MariaDB

El procedimiento descrito en el README de Docker crea un volcado SQL en el contenedor y lo copia al anfitrión:

```shell
docker compose exec db sh -c 'MYSQL_PWD="$MARIADB_ROOT_PASSWORD" mariadb-dump -uroot --single-transaction --quick --routines --triggers sigsm > /tmp/sigsm_backup.sql'

docker compose cp db:/tmp/sigsm_backup.sql ./sigsm_backup.sql

docker compose exec db rm /tmp/sigsm_backup.sql
```

8.3 Respaldo de documentos

```shell
mkdir storage_backup
docker compose cp backend:/var/www/api/storage/. ./storage_backup/
```

Guardar la copia SQL y el directorio de documentos con la misma fecha y referencia de versión. Protegerlos frente a acceso no autorizado, almacenarlos fuera del equipo principal y comprobar su integridad. Durante operaciones críticas, evitar modificaciones simultáneas que generen incoherencias entre la copia SQL y los archivos.

8.4 Automatización existente en el proyecto

El repositorio incorpora el script Segunda Entrega/Sistemas/scripts/backups.sh. Su función backupDocker exporta MariaDB mediante mariadb-dump, copia el almacenamiento del backend y genera un archivo comprimido sigsmDocker_FECHA.tar.gz. El script utiliza rutas específicas del servidor de desarrollo, por lo que deben revisarse DOCKER_DIR y BACKUP_DIR antes de trasladarlo a otro equipo. La política histórica del proyecto contempla respaldos locales diarios y remotos semanales; su ejecución efectiva y la configuración de cron deben verificarse en cada servidor. Docker Compose, por sí mismo, no programa respaldos.

8.5 Restauración

Restaurar información es una operación sensible: identificar el respaldo correcto, preparar una instalación compatible, conservar previamente el contenido actual si corresponde y restringir las escrituras durante el procedimiento. Para importar el SQL, la base sigsm debe existir.

```Shell
docker compose cp ./sigsm_backup.sql db:/tmp/sigsm_backup.sql
docker compose exec db sh -c 'MYSQL_PWD="$MARIADB_ROOT_PASSWORD" mariadb -uroot sigsm < /tmp/sigsm_backup.sql'
docker compose exec db rm /tmp/sigsm_backup.sql
```

Para restaurar los documentos y corregir los permisos:

```Shell
docker compose cp ./storage_backup/. backend:/var/www/api/storage/
docker compose exec -u root backend chown -R www-data:www-data /var/www/api/storage
```

Concluir comprobando el inicio de sesión, la consulta de datos y la visualización y descarga de PDF. Las restauraciones deben ensayarse previamente en un entorno distinto del de uso habitual.

9. Seguridad y acceso a la aplicación

9.1 Acceso local y por red

La instalación predeterminada publica el frontend en http://localhost:8080. Para acceder desde otra máquina de la red local se necesita la dirección IP del servidor, el puerto configurado y una regla de firewall que permita únicamente el tráfico autorizado. No es necesario publicar directamente el backend ni MariaDB. Si se quiere restringir el servicio al propio anfitrión, puede modificarse el mapeo de puertos del frontend para enlazarlo a 127.0.0.1.

9.2 Limitaciones del despliegue

La configuración actual utiliza HTTP y está orientada a entornos locales, educativos y de demostración. No debe exponerse sin protección a Internet ni usarse con información sensible. Un despliegue institucional requeriría HTTPS, configuración de red, controles de acceso, evaluación de seguridad, estrategia de respaldos y aprobación técnica del Departamento Técnico de Informática.

9.3 Protección de secretos y permisos

Utilizar contraseñas robustas y diferentes, limitar el acceso al archivo .env, mantener control sobre los administradores y sus roles, evitar datos reales durante pruebas y verificar los permisos del almacenamiento. La gestión de la cuenta inicial y sus contraseñas debe realizarse mediante los procedimientos autorizados por la aplicación y la base de datos, no simplemente editando variables de entorno ya inicializadas.

10. Resolución de problemas frecuentes

10.1 Docker no responde o la aplicación no abre

Ejecutar docker info y docker compose ps para comprobar el motor y los servicios. Revisar los logs de frontend y backend. Si el puerto 8080 se encuentra ocupado, cambiar WEB_PORT en .env y recrear el frontend mediante docker compose up -d --no-deps frontend.

```shell
docker info
docker compose ps
docker compose up -d --no-deps frontend
```

10.2 Error 502 en la API

Un error 502 puede indicar que Nginx no consigue conectarse al backend. Revisar docker compose ps y docker compose logs --tail=100 backend; comprobar que Apache esté activo y que los contenedores compartan la red de Compose.

```shell
docker compose ps
docker compose logs --tail=100 backend
```

10.3 Fallos de autenticación o admin-init

Revisar docker compose logs admin-init, la disponibilidad de MariaDB y las credenciales ingresadas. El administrador inicial necesita un correo válido y una clave de al menos 12 caracteres. Cambiar ADMIN_PASSWORD en .env no modifica la contraseña de un funcionario existente.

```shell
docker compose logs admin-init
```

10.4 MariaDB no saludable o esquema sin actualizar

Consultar docker compose logs db y revisar espacio, variables y volúmenes. Los scripts SQL de inicialización se ejecutan solo si la base es nueva. Para alterar una instalación con datos, aplicar migraciones comprobadas, nunca docker compose down -v.

```shell
docker compose logs db
```

10.5 Seed de demostración o PDF faltantes

Consultar docker compose --profile demo logs seed-demo. El seed exige una instalación limpia y no genera PDF. Si faltan documentos, revisar docker compose exec backend ls -la /var/www/api/storage, los metadatos en MariaDB y los permisos del directorio.

```shell
docker compose --profile demo logs seed-demo
docker compose exec backend ls -la /var/www/api/storage
```

10.6 Los cambios publicados no aparecen

Confirmar que los cambios estén en la rama main, reconstruir sin caché el servicio correspondiente, recrear su contenedor y consultar /etc/sigsm-commit para verificar qué versión quedó instalada.

11. Lista de comprobación final

- ☐ Docker, Compose y Git responden correctamente.
- ☐ Se creó .env con contraseñas seguras y sin publicar secretos.
- ☐ docker compose config --quiet termina sin errores.
- ☐ frontend, backend y db están en ejecución.
- ☐ La base de datos informa estado saludable.
- ☐ admin-init terminó correctamente.
- ☐ La aplicación abre en la dirección y puerto previstos.
- ☐ El administrador puede iniciar sesión.
- ☐ Funcionan las consultas principales y, si corresponde, los datos demo.
- ☐ Se identificaron y conservaron los volúmenes db_data y docs_data.
- ☐ Existe un respaldo conjunto de base de datos y documentos.
- ☐ Se registraron fecha, responsable y commit de la versión desplegada.

12. Referencias

Repositorio general del proyecto:
https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU

Manual de Docker del repositorio:
https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU/blob/main/prototipos/docker/README.md

Configuración Compose:
https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU/blob/main/prototipos/docker/compose.yaml

Ejemplo de variables:

https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU/blob/main/prototipos/docker/.env.example

Script de respaldos:

https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU/blob/main/Segunda%20Entrega/Sistemas/scripts/backups.sh